

프로젝트 전반 메커니즘 — 한 번에 훑기

날짜: 2026-05-01 상태: Stage 2 종료 시점 (1A·1A-2·1B·2 누적 93 체크 PASS)

§1. 앱 lifecycle / 오토로드

project.godot 의 autoload 6 개:

LimboConsole — 콘솔 addon
GameData — 상수 데이터
GameManager — 앱 코디네이터 (title ↔ run 전환)
RunManager — 회차·내부런 상태 (영속 단일 추적점)
BattleManager — 전투 시물 (자기완결, lifecycle 분리)
EventManager — 이벤트 시물 (RunManager 의 sub-system)

main.tscn 가 진입 씬 구조:

Main (game_ui.gd)
├─ run_ui (run_ui.gd) — phase 별 화면 dispatch
│ ├── battle_ui (battle_ui.gd)
│ ├── reward_screen
│ ├── result_screen
│ ├── research_screen (research_ui.gd)
│ └─ event_screen (event_ui.gd)
└─ title_screen — start/load/settings/quit 버튼

GameManager.app_phase ∈ {title, in_run} — game_ui 가 title_screen / run_ui 토글.

§2. 상수 데이터 (game_data.gd , 375 라인)

읽기 전용. 시물이 참조만:

상수	내용
BATTLE_RULES	hand_size_per_arm, energy_per_arm, body_arm_weight
INITIAL_BODY	히로인 max_hp + degradation stages
CARD_TEMPLATES	카드 12종 (정상 6 + 열화 6) — cost / effects / degradation_resistance
ARM_MODULES	팔 모듈 3종 — slot_type / max_hp / card_ids / degradation

상수	내용
RESEARCH_OPTIONS	회귀 직전 강화 옵션 3 — type / params / base_price
ENEMIES	몬스터 5종 — max_hp / intents / data_drop
TEST_MAP_GRAPH	6 노드 hex 사이클 — connections / position / type / enemy_id
EVENTS	이벤트 풀 7개 (intro / regression / repair_choice + chain 5)

삼항 §1 의 "상수 데이터" 결. 매직 넘버 흩어져있지 않음.

§3. 4 개 시물 매니저

3-1. GameManager (24 라인) — 앱 코디네이터

```
start_run()      → RunManager.init_run() + app_phase="in_run"
return_to_title() → RunManager.reset() + app_phase="title"
quit_app()       → tree.quit()
```

얕음. 진짜 상태는 RunManager 가 들고 있음.

3-2. RunManager (597 라인) — 영속 단일 추적점

두 dict 보유: - `big_run_data` — 큰 런 (= reset 전 전체 아크) 마스터. 회귀 통과해 유지. - `run_data` — 현재 내부 런 작업 사본. `big_run_data.duplicate(true)` + 맵·current_node 덮어쓰기.

스키마 핵심 필드:

```
phase: "map" | "battle_preview" | "combat" | "reward" | "research" | "lose" | "event"
body_hp / body_max_hp
arm_instances: { id → {instance_id, template_id, name, slot_type, max_hp, hp, cards,
equipped_arms: { L: id, R: id }
arm_inventory_max: 6
map: { id → node }
current_node_id
research_data      — 화폐 (적 드롭)
seen_events       — 이벤트 발생 이력 (chain root 만 카운트)
meta.big_run_count — 회귀 횟수
```

진입점 (public API): - `init_run()` / `reset()` / `_start_internal_run()` / `end_internal_run(result)` - `move_to_node(id)` — 인접 검사 + type 분기 (research / event / 일반) + phase 전이 - `start_combat()` — `BattleManager.begin_battle(input)` 호출 + phase="combat" - `equip_arm(side, id)` / `upgrade_card(...)` — 덱 조작 - `purchase(idx)` / `leave_research()`

— 연구 페이지 - `apply_event_action(action)` — Type 1 이벤트 액션 디스패처 (EventManager 가 호출) - `_apply_<type>(params)` — `body_boost` / `arm_attack_boost` / `arm_durability_boost` (research 와 event 공용)

시그널: `state_changed`, `internal_run_started`, `internal_run_ended(result)`.

핸들러: `_on_battle_ended(result)` — `body_hp` · `arm_hp` · `drop` 반영, `victory`→`reward` / `defeat`→`lose`.
`_on_event_resolved(result)` — `seen_events` 갱신 + `phase="map"`.

3-3. BattleManager (454 라인) — 자기완결 시물

battle_state dict 일시 보유:

```
body_hp/max, arm_l/r (deep-copied), enemy_id/name/hp/max/intents/intent_idx,  
deck/hand/discard, energy, block_l/r, turn, phase ("player"|"enemy"), result, drop
```

lifecycle: `begin_battle(input)` → 진행 → `battle_ended.emit(result)`. RunManager 의 `start_combat` 이 input 으로 `body_hp` / `arm_l` / `arm_r` 전달, `_on_battle_ended` 가 `result({result, body_hp, arm_l, arm_r, drop})` 받아서 반영.

삼항 §3 "독립 시물" 의 정석. 진행 중 RunManager 직접 read/write 0.

카드 효과 풀 (§7): `deal_damage` / `transfer_block` / `damage_own_arm`. 저하 (degradation) 적용 — `calc_drop × body_arm_weight × card_resistance`.

Snapshot 패턴: `get_snapshot()` 한 함수가 UI 가 필요한 모든 read-only 데이터 dict 반환. UI 는 dict 만 읽음.

3-4. EventManager (255 라인) — sub-system 시물

event_state dict — 이벤트(chain) 활성 동안 채워짐:

```
event_id, kind, phase, chain_root_id, line_idx (dialogue 만)
```

kind 풀: `dialogue` / `effect` / `choice` / (movie 스텝).

chain: `def.next` 가 있으면 `_finalize_event` 가 `_transition_to_event(next)` 로 phase 안 빠지고 자동 전이. `chain_root_id` 는 chain 의 첫 `begin_event` 에서 고정 — `seen_events` 카운트 키.

트리거 풀: `run_start` (이벤트 1개 — `intro_speech`), `node_enter` (`node_type` 매치).

RunManager 호출 (sub-system 결합): - `RunManager.big_run_data.seen_events` 직참조 (once_per 필터) - `RunManager.apply_event_action(action)` 직호출 (effect 적용)

(메모리 — EventManager = dependent sub-system, BattleManager 결 강제 X.)

시그널: `event_state_changed` (UI 구독), `event_resolved(result)` (RunManager 구독, chain 끝에 1 회).

§4. 시물 간 통신 — 두 패턴 공존

	BattleManager	EventManager
패턴	삼항 §3 정석 — input/result 만	sub-system — 직접 read/write OK
진입	<code>begin_battle(input)</code> — RunManager 가 모든 데이터 전달	<code>begin_event(id, _)</code> — context 안 씬, EventManager 가 RunManager 직참조
진행 중	RunManager 호출 0	seen_events 직읽기, apply_event_action 직호출
종료	<code>battle_ended(result)</code> 1회	<code>event_resolved(result)</code> 1회 (chain 끝)
정당화	전투 = 자기완결 미니게임, 외부 무관	이벤트 = run state 변경 + 표시 본업, 본질적 결합

§5. UI 층

삼항 §1-3 결 — 시물 읽기 + 명령 보내기, 자체 변이 0.

5-1. game_ui (42 라인)

title_screen / run_ui visibility 토글. 타이틀 버튼 → GameManager API.

5-2. run_ui (513 라인)

RunManager.state_changed 구독. `screens` dict 로 phase → 화면 매핑:

```
{"combat": battle_ui, "reward": ..., "lose": ..., "research": ..., "event": event_sc
```

phase == "combat" or "event" 시 `hide_overlay` (팔 인스펙터 / 히로인 일러스트 / 라벨 숨김). 맵 디스플레이 / 전투 프리뷰 / 회귀 카운트 라벨 / 연구 데이터 라벨 자체 빌드.

5-3. battle_ui (1048 라인) — 가장 큰 UI

mockup HTML 결의 Godot 이식. `BattleManager.get_snapshot()` / `get_card_preview(idx)` read, `play_card(idx)` / `end_turn()` 명령. 배경·히로인·적 스프라이트, 카드 손패 원호 배치, HP 바 셰이더 등.

5-4. event_ui (137 라인)

EventManager.event_state_changed 구독. kind 분기: - dialogue: 박스 + speaker/text + 클릭→advance_line - choice: 박스 + prompt + 동적 버튼 N→select_choice(idx) - 그 외: 무화면 (effect 즉시 통과)

5-5. research_ui (120 라인)

RunManager.state_changed 구독, phase=="research" 시 RESEARCH_OPTIONS 카드 N장 빌드.

`purchase(idx)` / `leave_research()` 호출.

§6. 데이터 lifecycle (4 단)

```
big_run_data      - _new_big_run() ~ reset()
                   (회귀 통과, 큰 런 단위)
    | duplicate(true)
    ↓
run_data          - _start_internal_run() ~ end_internal_run()
                   (내부 런 단위, 회귀 시 역류 + 재생성)
    | snapshot
    ↓
battle_state      - begin_battle() ~ _finalize_battle()
                   (전투 단위, 결과만 RunManager 로 반환)

event_state       - begin_event() ~ _resolve_chain()
                   (이벤트 chain 단위, 직접 결합)
```

역류 (회귀): `end_internal_run` 이 `run_data` 의 변경된 카드/장착 상태를 `big_run_data` 로 backflow → `_start_internal_run` 이 다시 duplicate.

§7. 트리거·이벤트 흐름

```
RunManager._start_internal_run
  ↳ EventManager.resolve_event("run_start", {})
    ↳ 매치 시 _begin_event_phase

RunManager.move_to_node(id)
  ↳ type "research" → _enter_research()
  ↳ EventManager._resolve_event_for_node(target)
    ↳ 매치 시 _begin_event_phase
  ↳ enemy_id != null → phase="battle_preview"
```

`_resolve_event_for_node = resolve_event("node_enter", {node_type: target.type})` .
`target.type` 이 무슨 값이든 ("event" , "repair" , 미래의 "event_combat" 등) EVENTS 풀에 매칭되는 트리거 있으면 발화.

§8. 연속 카운터

카운터	위치	갱신	용도
<code>meta.big_run_count</code>	<code>big_run_data</code>	<code>end_internal_run</code>	회귀 횟수 (<code>run_ui</code> 이상단 라벨)
<code>seen_events</code>	<code>big_run_data</code>	<code>_on_event_resolved</code> (chain root id)	<code>once_per</code> : "big_run" 필터
<code>research_data</code>	<code>big_run_data</code>	<code>_on_battle_ended</code> (drop) / <code>purchase</code> (- price)	강화 화폐

`reset()` 시 셋 다 소멸 (`big_run_data = {}`).

§9. 회귀 (regression) 흐름

플레이어가 노드 4 (type "research") 진입

- `run_data["research_offers"]` = 무작위 2개 옵션
- `phase = "research"`
- `research_ui` 노출

플레이어 옵션 0~2 회 적용 (`purchase`) - `research_data` 차감 + `_apply_<type>`

플레이어 "회귀" 버튼 (또는 `leave_research`)

- `end_internal_run("cleared")`
 - | `run_data`의 `cards` 변경분을 `big_run_data`의 `arm_instances`로 역류
 - | `big_run_data`의 `equipped_arms` 갱신
 - | `big_run_count += 1`
 - | `run_data = {}` → `internal_run_ended.emit`
 - ↳ `_start_internal_run`
 - | `run_data = big_run_data.duplicate(true)`
 - | `map` 새로 빌드, `current = 1`, `phase = "map"`
 - ↳ `run_start` 트리거 평가 (`intro_speech`는 `once_per`로 필터됨)

역류 대상: `cards`, `equipped_arms`. 화폐(`research_data`), HP 같은 건 그대로 (`big_run_data`가 `truth`).

§10. 테스트 인프라 (4 시나리오, 누적 93 체크)

```
test/  
  test_event_1a.{gd,tscn}  - dialogue 시물 (23) [--headless OK]  
  test_event_1a2.{gd,tscn} - run_start + 다중 (25) [--headless OK]  
  test_event_1b.{gd,tscn}  - UI 배선 + 스크린샷 (17)  
  test_event_2.{gd,tscn}   - effect/chain/choice + 스크린샷 (28)  
test_screenshots/         - 자동 캡처 PNG (.gitignore)
```

각 테스트 = `_ready` → `_run_scenario` → assertion 누적 → `quit(exit_code)`. 시뮬 단계 (1A/1A-2) 는 `--headless` 가능, UI/스크린샷 단계 (1B/2) 는 `--headless` 빼고 실행 (창 잠시 뜸).

§11. 코드량·구조 한 줄 요약

시뮬 (5 파일): 1330 라인 (`game_manager` + `run_manager` + `battle_manager` + `event_manager`)
UI (5 파일): 1860 라인 (`run_ui` + `battle_ui` + `event_ui` + `research_ui` + `game_ui`)
테스트 (4 파일): 634 라인

시뮬 / UI $\approx 1 : 1.4$

삼항 §1 평가: 시뮬 5 파일만 읽으면 게임 메커닉 100% 이해 가능 — UI 코드 안 봐도 됨. 자가 검증 통과.

§12. 잠재적 이슈 / 개선 거리 (참고만, 청소 X)

- `research_ui` ↔ `run_ui` 의 역할 분담 약간 모호 — 회귀 라벨이 `run_ui` 에, 연구 데이터 라벨이 `run_ui` + `research_ui` 양쪽에. 큰 문제 X.
- `battle_ui` 가 1048 라인 — 시각 자산이 많아 어쩔 수 없지만 분할 가능.
- `EventManager` 의 조회자 부재 — UI 가 `event_state.get(...)` 직접 파싱. `battle_manager` 의 `get_snapshot()` 결과 일관 격차. 콘텐츠 작성 시 점진 정리 가능.
- `CHARACTERS` 레지스트리 — `speaker freeform String` 이라 다국어·portrait 도입 시 일괄 lookup 필요해질 것.

이런 것들은 콘텐츠 작성 진행하면서 자연 발견되는 시점에 처리 권장.